

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\ a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf\_076beba8-59c\agent-ae420fcae0ba9ad99.jsonl`
- SHA-256: `428498bc92cab2b9c36ff9b9fddb3eea500fce8f25a9e15d9a6c81b321c7e17`
- Source modified: `2026-05-31T04:19:40+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf\_076beba8-59c`
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`

Transcript

1. user (2026-05-31T04:18:52.778Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):

1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMV3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:

- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:

****URL:**** <https://arxiv.org/html/2412.20662v2>

****Title:**** Enhancing Table Recognition with Vision LLMs: A Benchmark and Neighbor-Guided Toolchain Reasoner

****Found via:**** Borderless-table accuracy & financial-number hallucination/verification (contrarian/skeptical) search

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:18:55.120Z)

I'll fetch the source and extract key claims relevant to the research question.

3. assistant (2026-05-31T04:18:57.808Z)

I'll fetch the arXiv paper now.

4. user (2026-05-31T04:19:15.305Z)

Key Findings on Vision LLMs for Table Recognition

****Publication Details:****

- Title: "Enhancing Table Recognition with Vision LLMs: A Benchmark and Neighbor-Guided Toolchain Reasoner"

- Authors: Yitong Zhou, Mingyue Cheng, Qingyang Mao, Qi Liu, Feiyang Xu, Xin Li, Enhong Chen

- Year: 2024

- Venue: arXiv (2412.20662v2)

- Code: Open-sourced at <https://github.com/lqzxt/NGTR>

(1) Vision LLM Performance vs Traditional Methods

****Benchmark Results (TEDS metric):****

On high-quality images (SciTSR):

- GPT-4o: 91.07 TEDS
- Traditional method LGPMA: 95.08 TEDS
- Gap: "4.38 TEDS points lower than the leading baseline models"

On low-quality images (PubTabNet):

- GPT-4o: 84.80 TEDS
- Traditional method LGPMA: 94.63 TEDS
- Gap: "13.63 TEDS points" wider performance difference

****Key Finding:**** "VLLMs achieved competitive accuracy on specific datasets...however, a noticeable performance gap remains in favor of traditional models."

(2) Hallucination & Content Recognition Risks

The paper identifies critical weaknesses in cell content handling:

- "VLLMs tend to ignore cells lacking semantic content...this also limits their ability to effectively capture the structural information of tables with many blank cells"
- VLLMs demonstrate "excessive reliance on contextual cues, causing the model to be misled by its own reasoning" (Appendix C case study)
- On borderless/semi-structured tables: Models struggle with "atypical table structures, thereby resulting in erroneous hierarchical structural information"

(3) Benchmark Comparisons

****Tested Models:****

- Open-source: Phi-3.5-Vision, Llama-3.2-90B-Vision
- Closed-source: GPT-4o-mini, Qwen-VL-Max, GPT-4o, Gemini-1.5-Pro

****Performance on challenging wild dataset (WTW):****

- NGTR+GPT-4o: 52.03 TEDS-Struct
- Traditional LORE: 93.86 TEDS-Struct
- Finding: "VLLMs demonstrate suboptimal performance compared to traditional lightweight baseline methods"

****Bottleneck Analysis:****

- Blur and overexposure "lead to loss of text information...inaccurate fine-grained text recognition"
- "Skewed tables result in substantial decrease in accuracy"
- Underexposure "has little effect on table recognition performance"

(4) Neighbor-Guided Toolchain Reasoner (NGTR) Solution

The method addresses low-quality image input bottleneck through:

****Three Key Components:****

1. ****Toolkit Preparation:**** Five lightweight tools?border enhancement, image upscaling, noise reduction, binarization, detection/cropping
2. ****Similarity-based Neighbor Retrieval:**** "Images with similar features exhibit similar results after being processed by the same image preprocessing toolchain"?uses ORB algorithm with Hamming distance
3. ****Reflection-driven Tool Selection:**** "Step-by-step module enhances the interaction between the VLLMs and the target image"?iteratively accepts/rejects preprocessing operations

****Results with NGTR:****

- SciTSR improvement: +1.88 TEDS (GPT-4o with NGTR: 92.58 vs direct: 90.70)
- PubTabNet improvement: +10.57 TEDS (85.03 vs 74.46)
- WTW improvement: +12.02 TEDS-Struct (52.03 vs 40.01)

(5) Model-Specific Findings

****Lightweight models (Phi):**** "Demonstrates consistently low performance across all evaluations"

****Llama-3.2-90B:**** "Outstanding performance...surpassing GPT by 2.73 points and Gemini by 2.36 points" on PubTabNet structure tasks

****Model size effect:**** Smaller VLLMs "struggle to achieve the required recognition accuracy" and "often fail to produce parsable markup sequences"

****Critical Limitation Acknowledged:**** The approach "depends on the underlying toolkit" and performance degrades when "neighbor candidates do not sufficiently cover a wide range of scenarios."

5. assistant (2026-05-31T04:19:18.561Z)

The fetch succeeded with rich, relevant content. This is a primary research paper (arXiv) directly on point for the research question ? VLMs vs traditional pipelines on table recognition, including borderless tables and hallucination risks. Let me extract the falsifiable claims.

6. user (2026-05-31T04:19:28.101Z)

Structured output provided successfully

7. assistant (2026-05-31T04:19:39.971Z)

I've fetched and analyzed the arXiv paper. Summary of what I found:

****Source quality: primary**** ? peer-style academic research paper (arXiv 2412.20662v2, Dec 2024) from a research group (Zhou, Cheng, Mao, Liu, et al.), with open-sourced code at github.com/lqzxt/NGTR. Directly on-topic: it benchmarks Vision LLMs against traditional table-recognition specialists.

****Five falsifiable claims extracted****, all bearing on muneem's core decision (VLM vs deterministic/specialist pipeline for dense financial tables):

1. ****(central)**** On clean tables GPT-4o trails specialist LGPMA by 4.38 TEDS (91.07 vs 95.08); on noisier images the gap widens to 13.63 TEDS. VLMs are measurably less accurate than dedicated table-structure models.
2. ****(central)**** On hard in-the-wild tables (WTW), even NGTR+GPT-4o hits only 52.03 TEDS-Struct vs 93.86 for traditional LORE ? a huge gap, directly relevant to muneem's hardest cases (borderless/variable-layout statements).
3. ****(central)**** VLMs systematically drop blank/semantically-empty cells, corrupting structure ? an omission failure mode that threatens per-row financial integrity.
4. ****(supporting)**** VLMs are misled by their own reasoning/context ? a hallucination mode where output reflects priors rather than pixels.
5. ****(supporting)**** Small open-source VLMs (e.g. Phi-3.5-Vision) perform consistently poorly and often fail to emit parsable markup ? the consumer-GPU-friendly tier is the least reliable.

****Relevance to the research question:**** This is a strong skeptical/contrarian data point. It quantifies that VLMs underperform specialist table-recognition models (especially on hard borderless tables), and documents concrete hallucination/omission modes on table content ? strongly supporting muneem's design of using deterministic parsers as primary with balance-reconciliation as the machine-check, and treating any VLM as a flagged fallback whose output must be auto-validated. Note: the closed-source leaders here (GPT-4o, Gemini) are cloud/approval-gated, not local; the local-runnable models tested (Phi-3.5-Vision,

Llama-3.2-90B-Vision) were among the weaker performers, and Llama-3.2-Vision additionally carries license restrictions flagged in the broader research question. The structured output has been submitted.